Date: __/__/____ Assignment - 6 Page No.: ____

Problem Statement - Write a program to implement polynomial operations using Circular Linked List. The program should support
1) Create a polynomial by inserting terms
2) Display the polynomial in a readable mathematical form
3) Add two polynomials & display the result

Objectives -
1) To implement a polynomial representation using a CLL by dynamically creating & storing polynomial terms.
2) To perform fundamental polynomial operations such as creating, displaying & adding two polynomials using CLL techniques.
3) To understand the advantages of CLL in managing dynamic data structures & efficiently handling polynomial arithmetic by combining like terms.

Theory -
A polynomial is a mathematical expression made up of terms, where each term consists of a coef. & a variable raised to a non-negative integer exponent. A general polynomial can be represented as

$$P(x) = a_n x^n + a_{n-1} x^{n-1} + \dots a_1 x^1 + a_0$$

where $a_0, a_1, \dots a_n$ are coef. & n is the highest exponent of the polynomial.

Date : __/__/____ Page No.:

In a computer program, each poly. term be represented using two fields: coef. & exponent. Since the no. of terms in a p may vary, a dynamic data structure such a linked list is suitable for storing p terms.

A CLL is a variation of a linked list which the last node points back to the first node instead of containing a NULL pointer. This creates a circular structur The circular nature of the list is u for repeated traversal & processing of po terms w/o requiring a seperate NULL condition.

The creation operation inserts polynom terms into the circular linked list. Whi inserting, the program shald maintain the order of exponents & avoid storin unnecessary terms with a coef. of zero. If a term with an existing exponent is inse its coef. can be combined with the existing term.

Polynomial addition is performed by compari the exponents of terms from two polynomial If the exponents are equal, the coef. ar added & the resulting term is inserted into the resultant polynomial. If the expo

Date : __/__/____

Handwritten Creative Memory....

Page No.: _____

of one term is greater than the other, that term is directly copied to the resultant polynomial. This process continues until all terms from both polynomials are processed. Terms having a resultant coef. of zero are not included in the final polynomial.

Platform -

- 64-bit Open Source Linux
- Open Source C Programming tool like gcc

Pseudo Code -

- Create Polynomial
  - Start
  - Create an empty CLL
  - Read the no. of terms.
  - Repeat for each term:
    - Read the coef. & exponent
    - Create a new node
    - Store the coef. & exponent.
    - Insert the node
  - Make the last node point back to the first node.
  - Return the polynomial
  - Stop

- Display Polynomial
  - Start

Date : __/__/____ Page No.: __

- If the polynomial is empty, display
- Set the pointer to the first node.
- Repeat:
  - Display the coef. & exponen
  - Move the pointer to next node
- Stop when the pointer reaches first n
- Stop

- Add polynomial
- Start
- Create an empty CLL
- Set pointers to the first nodes of bo
- Compare the exponents
- If exponents are equal:
  - Add their coef.
  - Insert the resulting term in re
  - Move both pointers forward.
- Continue until all terms of both poly are processed
- Copy any remaining term into result
- Display the resultant polynomial
- Stop

Time Complexity -

- Create Polynomial - $O(n^2)$
- Display Polynomial - $O(n)$
- Add Polynomial - $O(n+m)$

Page No.:

Conclusion - The program successfully implements polynomial creation, display & addition using a CLL, demonstrating efficient dynamic memory management & polynomial manipulation.

FAQs

1) Why is a Circular linked list used to represent polynomials instead of a linear linked list?

A Circular linked list allows continuous traversal because the last node points back to the first node. It also supports dynamic memory allocation & efficient poly. operations.

2) What information is stored in each node of the Circular Linked list?

Each node stores the coef., exponent & a pointer to the next node. The coef. represents the numerical value, while the exponent represents the power of the variable.

3) What happens if one of the polynomials is empty during addition operation?

Date : __/__/____

Page No.:

- If one polynomial is empty, the re
is simply the other polynomial. I
both polynomials are empty, the re
is zero.

```c
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int coeff;
    int exp;
    struct Node *next;
};

struct Node* createNode(int coeff, int exp) {
    struct Node *newNode =
        (struct Node*)malloc(sizeof(struct Node));

    newNode->coeff = coeff;
    newNode->exp = exp;
    newNode->next = newNode;

    return newNode;
}

void insertTerm(struct Node **head, int coeff, int
exp) {
    if (coeff == 0)
        return;

    struct Node *newNode = createNode(coeff,
exp);

    if (*head == NULL) {
        *head = newNode;
        return;
    }
```

```
    struct Node *temp = *head;
    struct Node *prev = NULL;

    do {
        if (temp->exp == exp) {
            temp->coeff += coeff;
            free(newNode);
            return;
        }

        if (temp->exp < exp)
            break;

        prev = temp;
        temp = temp->next;
    } while (temp != *head);

    if (prev == NULL) {
        struct Node *last = *head;

        while (last->next != *head)
            last = last->next;

        newNode->next = *head;
        last->next = newNode;
        *head = newNode;
    } else {
        newNode->next = temp;
        prev->next = newNode;
```

```
struct Node* createPolynomial() {
    struct Node *head = NULL;
    int n, coeff, exp;

    printf("Enter number of terms: ");
    scanf("%d", &n);

    for (int i = 0; i < n; i++) {
        printf("Enter coefficient and exponent: ");
        scanf("%d %d", &coeff, &exp);
        insertTerm(&head, coeff, exp);
    }

    return head;
}

void displayPolynomial(struct Node *head) {
    if (head == NULL) {
        printf("0");
        return;
    }

    struct Node *temp = head;
    int first = 1;

    do {
        if (temp->coeff != 0) {
            if (!first && temp->coeff > 0)
                printf(" + ");
            else if (temp->coeff < 0)
```

```
            int c = abs(temp->coeff);

            if (first && temp->coeff < 0)
                printf("-");

            if (temp->exp == 0)
                printf("%d", c);
            else {
                if (c != 1)
                    printf("%d", c);

                printf("x");

                if (temp->exp != 1)
                    printf("^%d", temp->exp);
            }

            first = 0;
        }

        temp = temp->next;
    } while (temp != head);

    if (first)
        printf("0");
}

struct Node* addPolynomial(
    struct Node *p1, struct Node *p2) {

    struct Node *result = NULL;
    struct Node *temp;
```

```
    if (p1 != NULL) {
        temp = p1;
        do {
            insertTerm(&result, temp->coeff,
temp->exp);
            temp = temp->next;
        } while (temp != p1);
    }

    if (p2 != NULL) {
        temp = p2;
        do {
            insertTerm(&result, temp->coeff,
temp->exp);
            temp = temp->next;
        } while (temp != p2);
    }

    return result;
}

int main() {
    struct Node *p1, *p2, *sum;

    printf("Enter first polynomial:\n");
    p1 = createPolynomial();

    printf("\nEnter second polynomial:\n");
    p2 = createPolynomial();

    printf("\nFirst polynomial: ");
    displayPolynomial(p1);
```

```
    printf("\nFirst polynomial: ");
    displayPolynomial(p1);

    printf("\nSecond polynomial: ");
    displayPolynomial(p2);

    sum = addPolynomial(p1, p2);

    printf("\nSum of polynomials: ");
    displayPolynomial(sum);

    printf("\n");

    return 0;
}
```